Python Basics Note

> **CompletePythonreferenceguidewithdefinitions,operations, and examples**

1. Variable

Definition:

A variable is a name used to store a value in memory. It works like a container that holds data.

Example:

```
name = "Shanith"
age = 20
price = 99.5
```

2. Naming Styles

- **snake_case:** `student_ name`
- **camelCase:** `studentName`
- **PascalCase:** `StudentName`

3. Data Types

Definition:

Data types define the kind of data stored in a variable.

Type	Example	Descr iption
`int`	`10`	Whole number
`float`	`10.5`	Decimal number
`complex`	`2 = 3j`	Real = Imaginary
`str`	`"Hello"`	Text
`bool`	`True / False`	Logical values
`list`	`=1,2,3=`	Ordered, changeable
`tuple`	`=1,2,3=`	Ordered, unchangeable
`set`	`=1,2,3=`	Unordered, unique
`dict`	`{"name":"Shanith"}`	Key–value pairs

4. Numeric Types

```
a = 10      #int
b = 3.14    # float
```

```
c = 2 + 3j # complex
```

5. Strings

Definition:

A string is a sequence of characters written inside quotes.

```
text = "Hello Python"
```

Multiline String

```
msg = """This is
a multiline
str ing"""
```

Slicing

```
word = "Python"
print(word[0:3]) # Pyt
print(word[-3:]) # hon
```

6. String Methods

Method	Definition	Example
`upper()`	Converts to uppercase	`"hi".upper()` → `'HI'`
`lower()`	Converts to lowercase	`"HI".lower()` → `'hi'`
`str ip()`	Removes extra spaces	`" hello ".strip()` → `'hello'`
`replace(a,b)`	Replace substring	`"apple".replace("a","A")`
`split()`	Splits into list	`"a,b,c".split(",")`
`capitalize()`	Makes first letter capital	`'python'.capitalize()`
`title()`	Makes every word start with capital	`'hi world'.title()`
`find()`	Finds index of substring	`'apple'.find('p')`
`count()`	Counts occurrence	`'apple'.count('p')`
`startswith()`	Checks start of string	`'hello'.startswith('h')`
`endswith()`	Checks end of string	`'data.txt'.endswith('.txt')`
`isdigit()`	True if digits	`'123'.isdigit()`
`isalpha()`	True if letters only	`'abc'.isalpha()`

Concatenation

```
a = "Hello"
b = "World"
print(a + " " + b)
```

f-String

```
name = "Shanith"
age = 20
print(f"My name is {name}, I am {age} years old.")
```

7. Operators

Arithmetic

`+`, `-`, `*`, `/`, `//`, `%`, `**`

Comparison

`==`, `!=`, `>`, `<`, `>=`, `<=`

Assignment

`=`, `+=`, `-=`, `*=`, `/=`

Logical

`and`, `or`, `not`

Membership

`in`, `not in`

Identit y

`is`, `is not`

Bitwise

`&`, `|`, `^`, `~`, `<<`, `>>`

8. List

Definition:

A list is an ordered, changeable collection that can hold different data types.

```
fruits = ["apple", "banana", "cherry"]
```

Accessing Items

```
print(fruits[0]) # apple
print(fruits[-1]) # cherry
```

🔹 Changing Items

```
fruits[1] = "mango"
```

🔹 List Operations

Method	Description	Example
`append(x)`	Adds at end	`fruits.append("grape")`
`inser t(i,x)`	Inserts at position	`fruits.insert(1, "orange")`
`remove(x)`	Removes first matching item	`fruits.remove("apple")`
`pop(i)`	Removes by index	`fruits.pop(1)`
`clear()`	Removes all items	`fruits.clear()`
`sor t()`	Sorts ascending	`fruits.sor t()`
`reverse()`	Reverses order	`fruits.reverse()`
`len()`	Count of items	`len(fruits)`
`count(x)`	Count of x	`fruits.count("apple")`
`index(x)`	Returns position	`fruits.index("cher r y")`
`copy()`	Copies list	`new = fruits.copy()`
`extend(list2)`	Joins two lists	`fruits.extend(["kiwi"])`

🔹 Loop Through List

```
for f in fruits:
    print(f)
```

🔹 Slicing

```
pr int(fruits[1:3])
```

🟩 9. Tuple

Definition:

A tuple is an ordered, unchangeable collection.

```
colors = ("red", "green", "blue")
```

🔹 Accessing Elements

```
print(colors[0]) # red
```

Tuple Operations

Method	Descr iption	Example
`count(x)`	Counts x	`colors.count("red")`
`index(x)`	Returns index of x	`colors.index("green")`
`len()`	Length	`len(colors)`

Tuple to List Conversion

```
temp = list(colors)
temp.append("yellow")
colors = tuple(temp)
```

---

10. Set

Definition:

A set is unordered, unindexed, and stores unique values.

```
numbers = {1, 2, 3, 3}
print(numbers) # {1, 2, 3}
```

Accessing Items

Use a loop (no index).

```
for n in numbers:
    print(n)
```

Set Operations

Method	Descr iption	Example
`add(x)`	Adds one item	`numbers.add(4)`
`update([x,y])`	Adds multiple items	`numbers.update([5,6])`
`remove(x)`	Removes (error if missing)	`numbers.remove(2)`
`discard(x)`	Removes (no error)	`numbers.discard(10)`
`pop()` `clear()`	Removes random item	`numbers.pop()`
`len()`	Removes all	`numbers.clear()`
	Count of items	`len(numbers)`

Joining Sets

Operation	Descr iption	Example
union()	Combines both	A.union(B) or `A \
intersection()	Common elements	A & B A ☰
difference()	Items in A not in B	B A ^ B
symmetric_difference()	Not common items	

🔹 Frozen Set

Immutableset.

```
fs = frozenset({1,2,3☰☰
```

🟩 11. Dictionary

Definition:

A dictionary stores data in key–value pairs.

```
student = {"name": "Shanith", "age"☰ 20, "course": "BCA"}
```

🔹 Accessing Items

```
pr int(student["name"])
pr int(student.get("age"))
```

🔹 Adding / Changing

```
student["age"] ☰ 21
student["city"] = "Kozhikode"
```

🔹 Removing

Method	Descr iption	Example
pop(key)	Removes by key	student.pop("age")
popitem()	Removes last	student.popitem()
del dict[key]	Delete manually	del student["city"]
clear()	Remove all	student.clear()

🔹 Dictionary Methods

Method	Descr iption	Example
keys()	All keys	student.keys()
values()	All values	student.values()

Method	Descr iption	Example
items()	All pairs	student.items()
update(dict2	Merges two	student.update({"mark" 85
copy()	Copy dictionary	new = student.copy()
get(key)	Safe access	student.get("name")

Loop Through Dictionary

```
for key, value in student.items():
    print(key, ":", value)
```

Nested Dictionary

```
students = {
"s1": {"name": "Shanith", "age" 20,
"s2": {"name": "Anshid", "age" 21
}
print(students["s1"]["name"]) # Shanith
```

---

12. Conditional Statements

Simple if

```
x 10
if x 5
    print("Greater than 5")
```

If–Else

```
x 2
if x 5
    print("Big")
else:
print("Small")
```

Elif (Multiple)

```
x 5
if x 10
    print("Large")
elif x 5
    print("Equal")
```

```
else:
print("Small")
```

Short If (Ternary)

```
a = 10
b = "High" if a > 5 else "Low"
```

If with Logical Operators

```
if x > 0 and x < 10:
    print("Between 0 and 10")
```

13. Loops

For Loop

```
for i in range(5):
    print(i)
```

While Loop

```
x = 0
while x < 5:
    print(x)
    x += 1
```

Loop Control Statements

Statement	Definition	Example
`break`	Exit loop	`if i==3: break`
`continue`	Skip iteration	`if i==2: continue`
`pass`	Do nothing	`pass`

Nested Loop

```
for i in range(2):
    for j in range(3):
        print(i, j)
```

14. Comprehensions

List Comprehension

```
squares = [x*x for x in range(5)]
```

Dictionary Comprehension

```
d = {x: x*x for x in range(3)}
```

Set Comprehension

```
s = {x for x in range(5) if x % 2 == 0}
```

---

15. Summary Table

Type	Ordered	Changeable	Duplicate	Indexed	Example
List	✅ ✅ ❌	✅ ❌ ✅ ✅	✅ ✅ ❌	✅ ✅ ❌	`[1,2,3]`
Tuple	✅				`(1,2,3)`
Set					`{1,2,3}`
Dict			❌ (keys)	✅ (by key)	`{"a":1}`

Python Functions

Definition of a Function in Python

A**function** is a block of reusable code that performs a specific task. Instead of writing the same code again and again, you can put it in a function and call it whenever you need.

Key points:

- Functions help in **code reusability**.
- Functions make your code **organized** and **easier to read**.
- You can pass **inputs** to a function and get **outputs** from it.

Syntax of a Function

```
def function_name(parameters):
    """
    Optional: Docstring to describe what the function does
    """
    # code to execute
    return value # optional
```

- `def` → keyword to define a function
- `function_name` → name you give to the function
- `parameters` → input values the function can take (optional)
- `return` → the value the function gives back (optional)

Example 1: Simple Function Without Parameters

```
def greet():
    print("Hello, Shanith!")

greet() # Calling the function
```

Output:

```
Hello, Shanith!
```

Example 2: Function With Parameters

```
def add_numbers(a, b):
    return a + b

result = add_numbers(5, 3)
pr int(resul t)
```

Output:

```
8
```

Parameters vs Arguments

1. Parameter

A **parameter** is a variable listed in a function definition.

Think of it as a **placeholder** for the value the function will use when called.

Example:

```
def greet(name): # 'name' is a parameter
 print(f"Hello, {name}!")
```

Here, `name` is a **parameter**. It doesn't have a value yet—it will get one when the function is called.

2. Argument

An **argument** is the actual value you pass to the function when calling it.

Example:

```
greet("Shanith") # "Shanith" is the argument
```

Here, `"Shanith"` is the **argument** that is passed to the parameter `name`.

Types of Arguments

1. Using `*args` (Arbitrary Positional Arguments)

- `*args` allows you to pass **any number of positional arguments** ≣0, 1, 2, ...) to a function.
- Inside the function, `args` is treated as a **tuple**.

Example:

```
def add_numbers(*args):
    total ≣ 0
 for num in args:
        total += num
    return total

print(add_numbers(2, 3))       # 5
print(add_numbers(1, 2, 3, 4≣≣ # 10
print(add_numbers())           # 0
```

2. Keyword Argument

- **Keyword arguments** are arguments that are passed to a function **with a key=value syntax**.
- You can pass them **in any order** because the function knows which parameter each value is for.

Example:

```
def greet(name, age):
 print(f"Hello {name}, you are {age} years old.")

greet(age=20, name="Shanith") # Keyword arguments
```

Output:

```
Hello Shanith, you are 20 years old.
```

✅ Here, `age=20` and `name="Shanith"` are keyword arguments. The order doesn't matter.

3. Default Parameter Argument

- A **default parameter** is a parameter that has a **default value**.
- If you **don't pass a value**, the function will use the default.

Example:

```
def greet(name="Guest"):
 print(f"Hello {name}!")

greet()        # uses default value
greet("Shanith") # overrides default value
```

Output:

```
Hello Guest!
Hello Shanith!
```

✅ Default parameters make your function more flexible.

4. Arbitrary Keyword Argument (`**kwargs`)

- `**kwargs` lets you pass **any number of keyword arguments** (0, 1, 2, ...) to a function.
- Inside the function, `kwargs` is treated as a **dictionary**.

Example:

```
def greet_all(**kwargs):
 for name, message in kwargs.items():
        print(f"{name} says {message}")

greet_all(Alice="Hello", Bob="Hi", Shanith="Hey")
```

Output:

```
Alice says Hello
Bob says Hi
Shanith says Hey
```

`print` vs `return`

1. `print`

- `print()` **displays a value on the screen**.
- It **does not give anything back** to the program.
- Once printed, the value cannot be used later in calculations unless you save it separately.

Example:

```
def add(a, b):
    print(a + b) # just prints the result

result = add(3, 4)
print("Result:", result)
```

Output:

```
7
Result: None
```

✅ Notice `result` is `None` because `print()` does not return anything.

2. `return`

- `return` **gives a value back from a function**.
- You can **store it in a variable**, **use it in calculations**, or **pass it to other functions**.
- After `return` is executed, the function **stops**.

Example:

```
def add(a, b):
    return a + b # returns the result

result = add(3, 4)
print("Result:", result)
```

Output:

```
Result: 7
```

✅ Here, the function returned `7`, and we stored it in `result`.

Positional-Only and Keyword-Only Arguments

1. Positional-Only Argument (`/`)

- In Python, if you put a `/` in the function parameters, **all parameters before `/` must be passed as positional arguments**.
- You **cannot use keyword syntax** for them.

Example:

```
def position_only(hai, /): # 'hai' must be positional
    print(hai)

position_only("hello")       # ✅ Works
# position_only(hai="hello") # ❌ Error: cannot use keyword
```

Explanation:

`hai` is **positional-only**, so you must pass it like `position_only("hello")`.

2. Keyword-Only Argument (`*`)

- If you put a `*` in the function parameters, **all parameters after `*` must be passed as keyword arguments**.
- You **cannot pass them positionally**.

Example:

```
def keyword_only(*, name):
print(name)

keyword_only(name="hello") # ✅ Works
# keyword_only("hello")      # ❌ Error: must be keyword
```

Explanation:

`name` is **keyword-only**, so you must write `name="hello"`.

3. Mixed Positional and Keyword Arguments

- You can **combine positional-only and keyword-only arguments** in the same function using `/` and `*`.

Example:

```
def position_keyword_arguments(a, b, /, *, c, d):
    print(a + b + c + d)

position_keyword_arguments(1, 2, c=3, d=4) # ✅ Works
# position_keyword_arguments(a=1, b=2, c=3, d=4) # ❌ Error: a and b mu
st be positional
# position_keyword_arguments(1, 2, 3, 4)     # ❌ Error: c and d must be k
eyword
```

Explanation:

- `a, b` → **positional-only** (before `/`)
- `c, d` → **keyword-only** (after `*`)
- You must pass values according to these rules.

Output:

```
10
```

String Operations

1. Escape Characters

```
print("hello\nworld") # \n → new line
print("hello\tworld") # \t → tab/space
print('he is good boy \'good boy\'') # \' → single quote inside string
```

Output:

```
hello
world
hello    world
he is good boy 'good boy'
```

2. String Functions

```
a = "python programing"
print(len(a))        # Length of string → 18
print(a.capitalize()) # First letter capital → "Python programing"
print(a.title())     # First letter of each word capital → "Python Programing"
```

```
cap = "HELLO"
sma = "World"
print(cap.swapcase()) # Swap case → "hello"
print(sma.swapcase()) # Swap case → "wORLD"
```

3. Find / Index

```
cap = "HELLO"
a = "python programing"
print(cap.find("L")) # First occurrence of "L" → 2
print(a.index("python")) # Index of "python" → 0
print(a.index("t"))     # Index of "t" → 2
```

4. Startswith / Endswith

```
b = "Good Morning"
print(b.startswith("Go")) # True
```

```
pr int(b.endswith("ng"))    # True
```

5. Check Types

```
print("123".isdigit())    # True, all digits
print("abcd".isalpha()) # True, all letters
print("abc123".isalnum()) # True, letters+digits
print(" ".isspace())      # True, only spaces
```

6. f-strings

```
name = "shanith"
print(f"My name is {name}") # String interpolation
```

Output:

```
My name is shanith
```

Python `range()`

The `range()` function is used to **generate a sequence of numbers**, often used in **loops**.

Syntax

```
range(stop)
range(start, stop)
range(start, stop, step)
```

- **start** → where to start (default is 0
- **stop** → where to stop (not included)
- **step** → difference between numbers (default is 1

Examples

1. Only stop

```
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

2. Start and stop

```
for i in range(2, 6):
    print(i)
```

Output:

```
2
3
4
5
```

3. Start, stop, and step

```
for i in range(1, 10, 2
    print(i)
```

Output:

```
1
3
5
7
9
```

4. Reverse range

```
for i in range(5, 0, ▤1▤▤
    print(i)
```

Output:

```
5
4
3
2
1
```

Key Points

- `range()` **does not include the stop value**.
- Can be converted to a list:

```
print(list(range(5))) # ▤0, 1, 2, 3, 4▤
```

- Useful in **for loops** and **iterating sequences**.

Variable Scope

1. Local Variable

- A **local variable** is **defined inside a function**.
- It **can only be used inside that function**.
- It **disappears** after the function ends.

Example:

```
def my_function():
    x ▤ 10 # local variable
    print("Inside function:", x)

my_function()
# print(x) # ❌ Error: x is not defined outside
```

Output:

```
Inside function: 10
```

✅ `x` exists only **inside the function**.

2. Global Variable

- A **global variable** is **defined outside any function**.
- It can be **accessed inside or outside functions**.
- Use the keyword `global` if you want to **modify it inside a function**.

Example:

```
y = 20 # global variable

def my_function():
    print("Inside function:", y) # can access global variable

my_function()
print("Outside function:", y)
```

Output:

```
Inside function: 20
Outside function: 20
```

Modify Global Variable

```
z = 5

def update():
    global z
    z = 10 # modify global variable
    print("Inside function:", z)

update()
print("Outside function:", z)
```

Output:

```
Inside function: 10
Outside function: 10
```

Python Array

An **array** is a **collection of items of the same type** stored together.

In Python, we can use the `array` **module** to create arrays.

Example

```
from array import array

# Create an array of integers
arr = array('i', [1, 2, 3, 4, 5])
print(arr) # Output: array('i', [1, 2, 3, 4, 5])

# Access elements print(arr[0])
# first element [] 1 print(arr[-1])
# last element [] 5

# Add elements
arr.append(6)      # add at the end
arr.insert(2, 10[] # add 10 at index 2
print(arr)         # Output: array('i', [1, 2, 10, 3, 4, 5, 6])

# Remove elements
arr.remove(3)      # remove first occurrence of 3
arr.pop()          # remove last element
print(arr)         # Output: array('i', [1, 2, 10, 4, 5])
```

Docstring

A **docstring** is a **special string written at the beginning of a function, class, or module** to explain **what it does**.

- It helps others (and yourself) **understand the code**.

- Docstrings are enclosed in **triple quotes** `""" """`.

Example

```
def greet(name):
    """
    This function greets a person.
 It takes one parameter 'name' and prints a greeting.
    """
 print(f"Hello, {name}!")

greet("Shanith")
```

Advanced Function Concepts

1. Lambda Function

- A **lambda function** is a **small anonymous function**.
- It can have **any number of arguments**, but **only one expression**.
- Syntax:

```
lambda arguments: expression
```

Example:

```
square = lambda x: x**2
print(square(5)) # Output: 25
```

✅ This is the same as:

```
def square(x):
    return x**2
```

2. `map()`

- `map()` applies a function to **all items in a list**.
- Syntax:

```
map(function, iterable)
```

Example:

```
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, numbers))
print(squared) # Output: [1, 4, 9, 16]
```

3. `filter()`

- `filter()` selects items from a list that **meet a condition**.
- Syntax:

```
filter(function, iterable)
```

Example:

```
numbers = [1, 2, 3, 4, 5]
even = list(filter(lambda x: x%2[][]0, numbers))
print(even) # Output: [2, 4]
```

4. `reduce()`

- `reduce()` **applies a function cumulatively** to items in a list, **reducing it to a single value**.
- You need to **import it from `functools`**.
- Syntax:

```
from functools import reduce
reduce(function, iterable)
```

Example:

```
from functools import reduce

numbers = [1, 2, 3, 4]
```

```
sum_all = reduce(lambda x, y: x + y, numbers)
print(sum_all) # Output: 10
```

`sort()` vs `sorted()`

1. `sort()`

- `sort()` is a **method of lists**.
- It **modifies the original list** (in-place) and returns `None` .
- Can sort in **ascending or descending order** with `reverse=True` .

Example:

```
numbers = [5, 2, 8, 1]
numbers.sort()
print(numbers) # Output: [1, 2, 5, 8]

numbers.sort(reverse=True)
print(numbers) # Output: [8, 5, 2, 1]
```

✅ Key point: **Original list changes**.

2. `sorted()`

- `sorted()` is a **built-in function**.
- It **does not change the original list**; returns a **new sorted list**.
- Works on **lists, tuples, strings, or any iterable**.

Example:

```
numbers = [5, 2, 8, 1]
new_numbers = sorted(numbers)
print(new_numbers) # Output: [1, 2, 5, 8]
print(numbers)     # Original list unchanged → [5, 2, 8, 1]

# Descending order
print(sorted(numbers, reverse=True)) # Output: [8, 5, 2, 1]
```

Scope - LEGB Rule

1. Scope in Python

Python has**4 types of variable scope** (LEGB rule):

- **L** (Local (inside function)
- **E** (Enclosing (inside outer function)
- **G** (Global (outside all functions)
- **B** (Built-in (Python keywords/functions)

2. Enclosed (Nonlocal) Scope

- When a function is **nested inside another function**, the inner function can access variables from the **outer function**.
- To **modify the outer function's variable** from the inner function, use the `nonlocal` keyword.

Example

```
def outer():
    x = 10 # Enclosing variable

    def inner():
        nonlocal x # tells Python we want to use the outer x
        x += 5
        print("Inside inner:", x)

    inner()
    print("Inside outer:", x)

outer()
```

Output:

```
Inside inner: 15
Inside outer: 15
```

Recursion

1. Recursion

- **Recursion** is when a function **calls itself** to solve a problem.
- It usually has:

  Base case → stops the recursion

  Recursive case → the function calls itself

Example: Factorial using recursion

```
def factorial(n):
    if n == 0:      # base case
        return 1
    else:
        return n * factorial(n-1) # recursive case

print(factorial(5)) # Output: 120
```

How it works:

```
factorial(5) = 5 * factorial(4)
factorial(4) = 4 * factorial(3)
...
factorial(0) = 1
```

✅ Recursion is **easy to write** but can use **more memory** for large inputs.

2. Tail Recursion

- **Tail recursion** is a type of recursion where the **recursive call is the last operation** in the function.
- This allows some languages to **optimize memory**, but **Python does NOT optimize tail recursion**.

Example: Tail recursive factorial

```
def factorial_tail(n, result=1):
    if n == 0:
```

```
        return result
    else:
        return factorial_tail(n-1, n*result) # recursive call is last

print(factorial_tail(5)) # Output: 120
```

Difference from normal recursion:

- Normal: `return n * factorial(n-1)` → multiplication happens **after** recursive call returns.
- Tail: `return factorial_tail(n-1, n*result)` → recursive call **is last**, no extra operation afterward.